DEVELOPMENT STORY

Building a Next-Generation Recommendation System with AI Agent Teams

One Desktop GPU, a Team of Three, and the Record of AI Collaboration

"AI writes the code,
but design decisions remain with humans."

This document records the journey of building a 14-task, 7-expert
PLE+adaTT recommendation system — without infrastructure budget,
using a single desktop GPU and a team of AI agents.

AIOps PLE Platform

Project Background

Team Composition and Constraints

The project team consisted of three people: one data scientist serving as PM/Lead Architect and two engineers. There was no dedicated infrastructure budget, and the only GPU available for development was a single desktop-grade RTX 4070 (12GB VRAM) installed in a local PC.

> **Constraint Summary**
>
> - **Team size**: 3 members (PM/Lead Architect/Data Scientist 1 + Engineers 2)
> - **Infrastructure**: No dedicated GPU server. One RTX 4070 12GB (local)
> - **Budget**: No infrastructure procurement budget. AWS SageMaker spot instances used
> - **Objective**: Build a next-generation system to replace the existing ALS-based recommendation system

The Reality of Infrastructure Constraints

Support from the organization was virtually nonexistent. Requests for GPU resources were met with "there is nothing we can do," and all project-related expenses — AI tool subscriptions (Claude Code, Gemini, Cursor), AWS cloud costs (SageMaker Spot instances, S3 storage), peripherals, and meals — were covered personally by the PM/Lead Architect.

The data collection environment was equally challenging. Requests to migrate to Spark or Impala were denied, forcing the team to work within a HIVE-based bottleneck environment. To overcome this, parallel query logic was designed from scratch to maximize network bandwidth utilization.

The workspace was an unventilated area adjacent to the server room, without adequate cooling.

The two team members were not formally contracted employees but youth advisory supporters — recent graduates participating in the project while preparing for employment.

All of these constraints ultimately gave the design its identity.

Just as selection pressure in biological evolution drives species specialization, resource constraints acted as selection pressure on the architecture. The 12GB VRAM ceiling fundamentally blocked the lazy approach of "scale parameters for performance" and instead forced evolution toward "encode expressiveness through structural inductive biases." Seven heterogeneous experts each encoding a distinct mathematical perspective, feature engineering borrowing structural isomorphisms from eleven academic disciplines, lightweight expert designs that run in FP32 on a single desktop GPU — all of these are adaptations born from constraint.

Had a large GPU cluster been available, this architecture would likely never have been conceived. The team would have stacked seven Transformer-based heavy experts and brute-forced the problem with parameters — a forgettable approach. Without constraint, there would have been no identity.

On-Premises System Scale

The on-prem system was not a prototype but a production-scale system: 80+ Airflow DAGs, Champion-Challenger model competition, weekly automated retraining, 734D feature tensor, 16 simultaneous tasks, and 62 data table ingestion. Building a system of this scale with 3 people was itself a result of AI-augmented development.

Project Objective

The existing financial product recommendation system was based on ALS (Alternating Least Squares) collaborative filtering. The goal was to replace it with a multi-task deep learning recommendation system built on PLE (Progressive Layered Extraction) + adaTT (Adaptive Task Transfer) architecture. The system processes 14 tasks through 7 expert networks, explicitly modeling inter-task relationships.

Architecture Decision Journey

Arriving at the final architecture required exploring and rejecting several candidates.

The legacy system was ALS-based collaborative filtering. The first alternative considered was the **Black-Litterman model**. Originating from portfolio theory, this model combines expert views with market equilibrium via Bayesian updating. However, it was impossible to decompose "how much each model contributed" from the blended posterior distribution. In financial practice, recommendations must be explained to customers, branch staff, and regulators alike — Bayesian updating rendered individual model contributions opaque, making business explanation impossible.

The second alternative was a **model ensemble** — training N independent models and combining their outputs. This introduced N management points, N times the serving cost, and explanations like "MLP #3 contributed 28%" were meaningless in a business context. Both cost and explainability were problematic.

A critical reframing emerged from this process: rather than combining experts **outside** a model, what if we combined them **inside** a single model? This was the turning point toward PLE's heterogeneous expert architecture.

Task group design also involved trial and error. An initial attempt with **GMM cluster subheads** — K clusters × T tasks = $K \times T$ complexity explosion — was abandoned in favor of 4 Financial DNA task groups (product holding probability, next product, customer value, churn risk), stabilizing the structure.

Organizing AI Agents

Phase-by-Phase AI Tool Deployment

Different AI tools were strategically deployed at each stage of the project. The key was leveraging each tool's strengths through deliberate role assignment.

Phase A: Ideation (Gemini)

Gemini was used for initial concept exploration and brainstorming. Its broad knowledge base proved effective for rapidly scanning architecture candidates and comparing the pros and cons of various approaches. ALS replacement options, Black-Litterman exploration, and model ensemble approaches were all explored through dialogue with Gemini.

The greatest value emerged from cross-disciplinary feature ideation. Questions like "Can chemical kinetics describe spending behavior?" and "Is product adoption structurally equivalent to an epidemic?" were posed to systematically identify academic domains with structurally isomorphic problems to financial customer behavior. The PM/Lead Architect brought domain expertise (FRM certification, credit analysis experience) while Gemini provided cross-domain connections. Through this process, the concept of "structural isomorphism" naturally emerged as a central design principle.

Gemini's broad knowledge was optimal not for depth in any single technology, but for rapidly scanning "which field has already solved a similar problem." The design direction of importing features from 11 academic disciplines was established during this phase and became the foundation for all subsequent technical decisions.

Phase B: Technical Validation (Claude Opus)

Claude Opus was employed to develop ideation results into concrete architectures. It was focused on tasks requiring technical depth, such as loss function design demanding mathematical rigor, data leakage verification, and normalization pipeline design.

Each expert's feasibility was validated one by one: "Can HGCN work on MCC hierarchy?", "Is Mamba efficient enough for 17-month sequences?" Deep technical discussions with Opus covered PLE vs MMoE trade-offs, adaTT loss-level vs representation-level transfer comparisons, and other architecture-level analyses. Notably, the expert collapse problem in homogeneous MoE was first identified through conversation with Opus, which later led to the discovery of the NeurIPS 2024 sigmoid gate paper.

Opus was also effective at challenging assumptions. Its counterargument — "Is Black-Litterman really suitable?" — accelerated the pivot toward PLE. During this phase, 19 technical reference documents (.typ files) were co-authored with Opus, serving as design specifications that each agent would reference during the subsequent implementation phase.

Phase C: Code Environment Setup (Cursor)

GitHub-based code environment configuration, project structure design, and initial boilerplate generation were performed with Cursor. Its strength lay in fast code navigation and refactoring within an integrated IDE environment.

During this phase, 6 initial design documents (00-09 architecture specifications) were drafted, and the most important deliverable was the establishment of CLAUDE.md guardrails. The config-driven principle, separation of concerns, and leakage prevention rules — the "constitution" that all subsequent AI agents would follow — were established before a single line of code was written. This ordering was intentional: guardrails first, agent execution second.

Phase D: Parallel Implementation (Claude Code — Opus/Sonnet)

In the full implementation phase, each team member served as the "team lead" for their AI agents. Opus and Sonnet were operated in parallel within the Claude Code environment to implement different modules simultaneously. This was the most intensive phase, with three humans each leading AI agent teams.

The PM/Lead Architect's AI team deployed Opus for architectural decisions (PLE config, adaTT task groups, logit transfer design) and Sonnet for rapid code implementation (generators, adapters, pipeline runner). Critical debugging sessions occurred during this phase: detection of 3 label leakage cases, diagnosis of 4 FP16 NaN root causes, and GPU utilization optimization. Engineer 1′s AI team handled the data ingestion pipeline, HIVE parallel query logic, feature engineering for 10 generators (TDA, HMM, Mamba, Graph, GMM, etc.), and the feature-to-business reverse-mapping registry. Engineer 2′s AI team was responsible for model training, mathematical verification, and knowledge distillation (PLE to LGBM).

The ability to maintain consistency across parallel workstreams was thanks to the CLAUDE.md guardrails established in Phase C and the interface contract verification process. After every parallel work session, it was mandatory to verify that the keys saved by File A matched the keys read by File B, preemptively preventing interface mismatches that could arise during integration.

Phase E: Experimentation + Papers (Claude Code Extension)

During the ablation experimentation phase, Claude Code was used as a real-time monitoring tool. Ablation progress, GPU utilization, and error detection were performed in real time. The PLE toggle bug was discovered through live debugging — the `use_ple=false` setting was altering the expert composition itself, making fair comparison impossible.

Literature research during experiment wait times also proved valuable. While observing that PLE's val_loss failed to converge, a hypothesis was formed through dialogue with Opus that the softmax gate's competitive nature was hindering convergence among heterogeneous experts. This led to finding the NeurIPS 2024 sigmoid gate paper, providing theoretical grounding. Experiment result analysis and paper writing proceeded simultaneously.

In the paper writing phase, 4 papers (English/Korean), and 22 technical documents were generated and refined through iterative work with Claude. The development story itself was written by reflecting on the project process together with Claude. In this phase, AI served not as a mere text generator but as a thought partner in constructing the meaning of the project.

Documentation Production Scale

AI collaboration extended far beyond code implementation. The on-prem project alone produced 260+ documents (28 design specs, 19 tech references, 16 code reviews, 95 reports, 5 guides), totaling 30+ MB of technical documentation. Many were co-authored with AI or drafted by AI then reviewed by humans. Notably, the "Sonnet Work Verification Report" demonstrates an AI-to-AI review process where Opus verified Sonnet's code output, and the "500+ Item Checklist for Claude Code Opus" shows systematic delegation of quality assurance tasks to AI agents.

Memory Bank and Guardrail System

The AI management framework established in the on-prem project was directly ported to the AWS project. A memory-bank system (8 context files: projectbrief, activeContext, progress, techContext, productContext, systemPatterns, tasks, style-guide) maintained cross-session context. .claude/RULES.md enforced coding rules, synchronized with .cursorrules so Cursor AI and Claude Code followed identical guardrails. The team even ran automated experiment branches across three AI platforms (exp/claude-auto-*, exp/codex-auto-*, exp/vertex-auto-*), comparing Claude, Codex, and Vertex AI.

Why Claude Code Was Essential

What proved decisive for this project's complexity was long context retention (1M tokens), cross-session memory banks, and parallel sub-agent execution.

Tracing 3 label leakage cases in sequence was only possible by maintaining context across days of work. After fixing the first leakage (has_nba duplicate column), the second (ground truth glob sorting) and third (generator label input) were discovered in the same session because the context of previous fixes remained alive.

Simultaneously diagnosing 4 FP16 NaN root causes (CGC entropy, OT Sinkhorn, Causal DAG, logits) required surveying the entire model architecture while tracing numerical operations in each expert. This was impossible with a file-by-file approach.

Discovering the NeurIPS 2024 sigmoid paper during experiment wait time, connecting it to our observation of PLE softmax non-convergence, and implementing the sigmoid gate — this all happened within one continuous context: experiment analysis to literature search to theory connection to code implementation in a single flow.

Maintaining consistency across 4 papers and 22 technical documents while making simultaneous edits was also a task only possible with an agent that remembered the entire document ecosystem.

Key Patterns Discovered in AI Collaboration

Throughout the project, recurring patterns of AI collaboration emerged. These patterns were not deliberately designed but manifested naturally through actual work.

> **1. "AI does HOW, humans decide WHAT and WHY"**
>
> : AI generates code and text, but architectural decisions are made by humans. The structural isomorphism insight emerged from human-AI dialogue, but the decision to adopt it as a design principle was a human judgment.
>
> **2. "Guardrails before agents"**
>
> : CLAUDE.md was written before code, not after. Just as a constitution precedes legislation, guardrails precede agent execution.
>
> **3. "Heterogeneous AI = heterogeneous experts"**
>
> : The model's heterogeneous expert design was directly applied to development tool selection. Each AI tool performed a specialized role, achieving quality and speed unattainable by any single tool.
>
> **4. "Memory bank for continuity"**
>
> : A persistent file system for session-to-session context preservation was the key mechanism for overcoming AI agents' greatest weakness: context loss between sessions.
>
> **5. "Fail fast with AI"**
>
> : Bugs that would take days to find manually — leakage, FP16 NaN, ablation filter failures — were detected and fixed within minutes with AI agents. Fast failure led to fast learning.
>
> **6. "AI as thought partner, not code machine"**
>
> : The project's critical intellectual breakthroughs — the structural isomorphism insight, the sigmoid gate hypothesis, identifying the expert collapse problem — all emerged from human-AI dialogue.

Phase	AI Tool	Role
A. Ideation	Gemini	Concept exploration, architecture scanning, cross-disciplinary brainstorming
B. Technical Validation	Claude Opus	Mathematical verification, loss design, leakage analysis, 19 technical docs co-authored
C. Environment Setup	Cursor	GitHub structure, CLAUDE.md guardrails, 6 design documents
D. Parallel Implementation	Claude Code (Opus/Sonnet)	3-person x AI team parallel coding, debugging, 10 generators
E. Experimentation + Papers	Claude Code Extension	Real-time monitoring, literature research, 4 papers + 22 technical docs

Quality Management Strategy

CLAUDE.md Guardrails

To ensure the quality of AI-agent-generated code, a CLAUDE.md file was placed at the project root to establish guardrails. This file serves as system instructions that the AI agent reads at the start of every session.

> **Core CLAUDE.md Guardrails**
>
> - **Config-Driven Principle**: All parameters are read from YAML. No hardcoding.
> - **Separation of Concerns**: Adapter handles only data transformation, Runner handles only the pipeline, train.py handles only training.
> - **Data Leakage Prevention**: Scaler is fit only on the train split. gap_days mandatory for temporal splits.
> - **4-Stage Code Review**: Compilation check → Interface contract verification → Hardcoding scan → Separation of concerns audit.

Memory Bank

AI agents lose context when a conversation session ends. To address this, a “memory bank” system was introduced. Session progress, design decision rationale, and feedback history were managed as structured markdown files, enabling the AI to quickly restore prior context at the start of each new session.

Interface Contract Verification

The greatest risk when parallel agents simultaneously modify different modules is interface mismatch — where the key names saved by File A differ from the key names read by File B. To prevent this, an "interface contract verification" step was mandatory after all parallel work. Cross-file key/field mapping tables were auto-generated to detect inconsistencies before they reached production.

Technical Challenges and Solutions

Over 20 technical issues arose during development. They are organized here into five categories. Rather than a debug log, each category illustrates a distinct engineering dimension required for large-scale multi-task training on desktop GPUs.

Data Integrity

Data contamination renders model performance meaningless. Label leakage and schema inconsistencies occurred repeatedly in this project, each requiring systematic detection and the construction of defensive guardrails.

Three Label Leakage Cases

Abnormally high performance (AUC = 1.0) was observed early in training, uncovering three leakage sources.

> - **Duplicate column**: `has_nba_1` existed as a perfect label duplicate (correlation = 1.0). Resolved by EXCLUDE before label re-derivation.
> - **File loading order**: Glob's alphabetical sorting loaded `ground_truth.parquet` instead of `benchmark.parquet`. Resolved by moving the file to a subdirectory.
> - **Generator input contamination**: GMM and other generators used label columns as input. An auto-exclude mechanism for `label_cols` was added.

A LeakageValidator was added as a mandatory pre-training step, and validation rules were documented in the CLAUDE.md guardrails to prevent recurrence.

apply_ablation Schema Not Updated

The ablation filter successfully removed features from tensors, but `feature_schema["columns"]` still listed the original 316 columns, so the model received identical 316-dimensional input every time. The fix required `apply_ablation` to simultaneously update `columns`, `num_features`, and `feature_group_ranges` alongside tensor modification.

Numerical Stability

Mixed precision training doubles throughput, but the narrow representable range of FP16/BFloat16 triggers NaN propagation. Four underflow incidents and two conversion errors occurred.

FP16 Underflow and NaN Propagation

During Phase 2 AMP training, FP16 underflow in CGC entropy, OT Sinkhorn, Causal DAG, and logit computations caused NaN to propagate. Small gradient values fell outside the FP16 representable range.

BFloat16 NumPy Conversion and GradScaler

NumPy does not support BFloat16 as a dtype, so `.numpy()` calls on BFloat16 tensors failed and all validation metric calculations broke. A `.float()` cast before every `.numpy()` call was standardized across the pipeline. Additionally, when every batch in Phase 2 produced NaN, no backward pass was called, and `scaler.step()` raised "No inf checks were recorded." A backward-count guard was added to skip the step when the count was zero.

Infrastructure

On a single desktop GPU, driver conflicts, background processes, and network restrictions can reshape the entire experimental design.

Docker GPU Passthrough and Zombie Containers

Docker GPU passthrough on Windows proved unstable due to CUDA version mismatches, leading to a switch to local Python development. Separately, unterminated zombie containers occupied GPU memory and reduced training speed to one-third of normal.

torch CPU/CUDA Version Conflicts

Installing SageMaker SDK v3 (3.7.0) silently replaced torch with a CPU-only build, disabling GPU training entirely. Pinning to SageMaker v2 (2.257.1) resolved the issue. In conda, CPU/CUDA build conflicts also recurred; the cache was purged and the environment rebuilt with explicit CUDA version pinning.

torch Cache Recovery and Ollama GPU Occupation

The government office firewall blocked `download.pytorch.org` (403 Forbidden). A cached `pytorch-2.5.1-cuda12.1` package in conda's `pkgs/` directory was manually copied to restore the environment. Separately, Ollama auto-started and consumed 2GB of VRAM. On a 12GB card, a single background process directly constrained batch size choices.

VRAM Spillover Analysis

At batch 6144: 12GB dedicated + 11GB shared GPU memory = 23GB total, 10 hours per scenario. At batch 2048: 9GB + 0.1GB = 9.1GB, 2 hours per scenario. Shared GPU memory traverses PCIe at 10–20x slower than dedicated VRAM. This quantitative analysis determined the optimal batch size.

Pipeline Engineering

System-level issues in large-scale data processing and ablation orchestration.

Transition from pandas to DuckDB/cuDF

Processing 941K rows with pandas caused memory spikes. Migrating to DuckDB (CPU columnar) and cuDF (GPU) simultaneously improved memory efficiency and throughput.

NVIDIA Merlin Ecosystem Evaluation and Selective Adoption

We initially attempted to adopt the NVIDIA Merlin ecosystem (NVTabular, HugeCTR, etc.) as a full-stack solution. However, our custom heterogeneous expert architecture — 7 distinct experts (DeepFM, Temporal Ensemble, HGCN, PersLay, LightGCN, Causal, OT) — did not fit well with Merlin's opinionated pipeline. Merlin is optimized for single-model training workflows and could not easily accommodate an architecture where each expert demands different input formats and computation graphs. In the end, we retained only Merlin's DataLoader component, adopted cuDF directly for GPU-accelerated preprocessing and feature engineering, and introduced Triton Inference Server for model serving and deployment. This decision reflects a practical engineering philosophy: evaluate full-stack frameworks thoroughly, but only adopt the components that genuinely fit.

Subprocess Pipe Deadlock

The ablation orchestrator used `subprocess.run(capture_output=True)`, but massive stdout exceeded the 64KB pipe buffer, causing a classic deadlock. Redirecting stdout/stderr to files resolved the issue.

Ground Truth File Loaded by Mistake

Glob's alphabetical ordering loaded `benchmark_ground_truth.parquet` before the actual source data, causing Phase 0 to build features from answer columns. Moving the ground truth file to a subdirectory excluded it from the glob pattern.

Batch Size Mismatch and bash JSON Escaping

`pipeline.yaml` specified batch_size=2048, but `run_ablation_manual.sh` overrode it to 6144, triggering VRAM spillover. All settings were unified to a single config source. A separate bash JSON escaping failure was also fixed.

Architecture Insights

Ablation experiments and the training process yielded fundamental discoveries about model structure.

PLE Toggle Bug and Ablation Filter Failure

With `use_ple=false`, all 7 heterogeneous experts collapsed into a single MLP, making the baseline comparison unfair. The fix preserved the expert basket and disabled only PLE layering. Additionally, `feature_group_ranges` stored only column-level keys, so the ablation filter's group-level matching never succeeded — all 24 scenarios showed identical AUC (0.913). Adding group-level keys resolved it.

GPU Utilization Optimization

Initial GPU utilization was low with batch size 512. Increasing batch size, tuning DataLoader parameters (num_workers, pin_memory), and pre-saving tensors in Phase 0 improved training throughput. However, the 12GB VRAM constraint capped batch size at 2048 — exceeding this caused spillover into shared GPU memory, degrading rather than improving performance.

Softmax vs Sigmoid Gate Discovery

PLE's val_loss froze at 3.702 in Phase 2, while shared_bottom (1 MLP) paradoxically outperformed ple_only (7 experts). The CGC softmax gate's competitive nature hindered convergence among heterogeneous experts. A NeurIPS 2024 paper confirmed the theoretical superiority of sigmoid gates; implementation is in progress. Gate function selection proved decisive for heterogeneous expert architectures.

Five adaTT Porting Bugs — Discovered by Comparing Against On-Prem Code

Background: In the structure ablation, adaTT consistently degraded performance (sigmoid: –0.006, softmax: –0.021, no PLE: –0.004). The problem was not adaTT's design but implementation errors introduced during porting.

Root Cause: A line-by-line comparison against the on-prem (gotothemoon) source code revealed five bugs.

1. **Gradient extraction frequency**: AWS extracted gradients only at the last batch of each epoch (once per epoch). The on-prem version extracted every 10 steps (17 times per epoch). Fixed by replacing the `_is_epoch_end_step` flag with `global_step % grad_interval`.
2. **Config load path**: The root-level `adatt:` section in pipeline.yaml was not being read from `model_config` or `label_schema`. Fixed by adding a `config.get("adatt")` fallback.
3. **freeze_epoch not passed**: `AdaTTConfig` was constructed without passing `freeze_epoch`, leaving it always None. Transfer weights kept adapting unstably until the end of training.
4. **Loss composition**: The on-prem version applies uncertainty weighting first (normalizing loss scales) and then adaTT transfer. The AWS version used either/or logic, silently disabling uncertainty weighting whenever adaTT was active. With 14 tasks having mismatched loss scales, transfer was dominated by the largest-loss tasks.
5. **warmup_epochs: 0**: Transfer began immediately while the affinity matrix was still identity (no measurements taken), resulting in meaningless loss sharing from epoch one.

Outcome: sigmoid_adatt AUC improved from 0.5605 to 0.5746 (+0.014). At peak (Ep6) it reached 0.5786, surpassing the sigmoid baseline (0.5771).

Lesson: Preflight logging (`"AdaTT config: warmup=X, freeze=X, source=X"`) was added so that config application can be verified before training begins. Had MLflow been in place, a significant portion of this investigation could have been avoided.

Design Philosophy: Where Is the Science?

"The Audience of Persuasion Is Always a Person"

The end consumers of a financial recommendation system are not algorithms but people. Customers ask "why this product?", branch staff must explain recommendation rationale, and regulators verify the model's decision-making process. A single probability value cannot persuade any of these three groups. Therefore, the criterion for every design decision was: "Can we explain the reason?"

The 2-Axis Decomposition Framework

The core structure of the architecture is based on a 2-axis decomposition: **Financial DNA** (who is the customer?) × **Data Modality** (what form is the data?). The Financial DNA axis consists of 4 task

groups: product holding probability, next product, customer value, and churn risk. The Data Modality axis is composed of heterogeneous experts corresponding to 5 feature types: state, snapshot, time-series, hierarchy, and item. The intersection of these two axes defines the entire model's learning structure.

The Expert Collapse Problem and the Need for Heterogeneous Experts

When homogeneous MLP experts (e.g., identically structured 3-layer MLPs) are used, **expert collapse** occurs during training — the gating network selects only one expert while the rest go effectively unused. This problem has been confirmed in large-scale experiments at Pinterest and Kuaishou. Structurally different heterogeneous experts (LightGCN, Causal OT, TDA, GMM, etc.) operate on different input spaces with different computational methods, making it impossible for them to converge to identical representations — structurally preventing collapse.

The Dual Role of Features: Prediction Material and Explanation Vocabulary

Features are not merely inputs for prediction performance; they also serve as vocabulary for explaining recommendation rationale. Even a feature with marginal AUC contribution holds irreplaceable value as explanation vocabulary if it enables statements like "this customer's consumption entropy is high, indicating diverse product experience" at the branch level. For this reason, features were never removed solely based on predictive performance criteria.

From Economics to Data Science

The design philosophy of this project originates from the PM/Lead Architect's intellectual journey. Trained in economics and decision science, then moving through financial engineering to data science, a fundamental question emerged: **"Where is the science in data-driven methodology?"**

Economics has accumulated centuries of scientific methodology — hypothesis formulation, theoretical frameworks, falsifiability. But economics itself borrowed tools from other disciplines: general equilibrium theory drew from thermodynamic equilibrium in physics, game theory from combinatorics in mathematics, econometrics from statistics. Economics is not uniquely scientific; rather, **cross-disciplinary tool transfer is a universal pattern of scientific progress**. Many Nobel laureates in economics came from physics and mathematics — Samuelson (thermodynamics to economic equilibrium), Black-Scholes (heat equation to option pricing), Nash (fixed-point theorem to game theory), Mandelbrot (fractals to financial volatility). Economics' most powerful tools came from other sciences.

But moving from financial engineering to data science, scientific rigor gradually faded. Machine learning models were at least mathematically transparent — OLS minimization, SVM margin maximization, decision tree information gain — one could explain *why* they work.

Deep learning changed this. "Neural networks" is a metaphorical name rather than a deep study of actual neural architecture, and the answer to "why does this weight have this value?" is merely "the data trained it that way." Science requires explanation. An entire field — philosophy of science — emerged to discuss falsifiability, paradigm shifts, and degenerating research programs. Current deep learning approaches felt more like *engineering than science*.

Structural Isomorphism: Bringing Science Back

The answer to this question was *structural isomorphism*.

Humanity has already discovered powerful scientific methodologies across disciplines over centuries. Chemical kinetics, epidemiology, information theory, topology — each represents frontier knowledge about what insights and causal relationships can be extracted from given phenomena.

If we properly recognize the structure of our problem (understanding financial customer behavior), we can import solutions from other disciplines that have already solved structurally equivalent problems. Just as Shannon brought Boltzmann's thermodynamic entropy to information theory, and Black-Scholes derived option pricing from the heat diffusion equation.

This is the fundamental motivation behind introducing features from 11 academic disciplines and designing heterogeneous experts specialized in each discipline's mathematical tools. Not "let's create many features," but **"what scientific questions should we ask?"** was the starting point of design.

The Place of Scientific Methodology in Recommendation

This project demonstrates that recommendation systems can be grounded in *scientific understanding* rather than mere pattern matching. Not every recommendation needs to end at "similar people bought this" (correlation); it can reach "this customer's consumption dynamics indicate this product is appropriate" (causal explanation).

Pearl's causal inference, Friedman's Permanent Income Hypothesis, Boltzmann's statistical mechanics — these scientists answered "why?" in their respective fields. Our heterogeneous expert architecture brings their tools to financial recommendation through structural isomorphism, attempting to endow recommendation systems with scientific explainability.

Key Lessons

"AI Writes the Code, but Design Decisions Remain with Humans"

AI agents generate code at remarkable speed, but architecture decisions, data leakage judgments, and cost optimization strategies depend on human domain knowledge and experience. AI excels at "how," but "why" and "what" must be defined by humans.

> *"The most dangerous moment is right after AI produces 'plausible-looking code.'*
> *If you stop critically reviewing at that point, technical debt accumulates."*

"AI Coding Without Guardrails Creates Technical Debt"

When AI is given free rein to code without a CLAUDE.md, hardcoded values proliferate, concerns become entangled, and untestable structures emerge. Guardrails do not restrict AI's productivity — they channel it in the right direction.

"The Mixture-of-Experts Philosophy Applies to Development Methodology Too"

The core philosophy of the PLE architecture — Mixture of Experts — was applied to the development methodology itself. Gemini specialized in broad exploration, Opus in deep analysis, Cursor in rapid environment setup, and Claude Code in implementation. Assigning roles aligned to each tool's strengths proved more effective than having a single AI tool do everything.

Results

System Built

Recommendation System

- 14-task multi-task learning
- 7-expert PLE network
- adaTT adaptive inter-task transfer
- Uncertainty weighting (Kendall et al.)
- Three logit transfer modes

Infrastructure and Experimentation

- 24 ablation scenarios
- AWS SageMaker spot instances
- Phase 0 (CPU) + Phase 1~2 (GPU) separation
- Config-driven pipeline architecture

Documentation

A total of nine technical documents were produced throughout the project: architecture overview, pipeline guide, expert details, feature reference, PLE+adaTT reference, Causal OT reference, distillation reference, temporal reference, and regulatory framework — all authored in Typst.

Papers

Two papers are being prepared for submission to arXiv, covering the experience of building a large-scale multi-task recommendation system under resource constraints, and the development methodology of a small team leveraging AI agents. This may be the first arXiv publication by a practitioner at a Korean financial institution.

Expert Specialization Revealed by Ablation

Analysis across 24 ablation scenarios clearly demonstrated task-type-specific expert specialization. LightGCN showed the greatest contribution for multiclass tasks (next product prediction), while the Causal expert excelled at regression tasks (customer value estimation). This empirically validates the heterogeneous expert design.

On-Premises Operational Results

The on-prem system achieved 85% compliance with the FSS AI RMF (11 full + 9 partial out of 24 items), with remaining gaps being organizational decisions rather than technical implementations. 12 regulatory compliance modules were implemented: AI notification, right to refuse, human reprocessing, fairness monitoring, conflict-of-interest prevention, herding detection, prompt injection defense, safety documentation, model cards, audit trail, consent management, and quality monitoring.

Evaluation Metric Framework

Gold standard metrics were established by task type: AUC for binary classification, Macro F1 for multiclass classification, and MAE for regression. This prevents the error of comparing all tasks with a single metric and ensures rigorous evaluation aligned with each task's characteristics.

Ops/Audit Agents — "AI Analyzes, Humans Decide"

The most ambitious design challenge in this project was building an autonomous pipeline diagnostic agent system, separate from recommendation reason generation. Core question: "Can a small team maintain regulation-compliant AI operations without dedicated MLOps staff?"

Design Process: From Conversation to Architecture

The design evolved progressively through continuous dialog in a single session:

1. **"Let's add agents"** → Ops/Audit separation → async decoupling rationale (latency, regulatory independence, fault isolation)
2. **"What model level?"** → "Rule engine handles 95%" → deterministic engine + LLM dialog interface separation
3. **"On-prem?"** → "Core features work without Bedrock" → on-prem baseline + AWS Bedrock extension
4. **"Hallucination risk"** → 3-agent independent voting → **"Delphi has convergence bias"** → 2-Round hybrid (independent voting + sequential deliberation) + minority report preservation
5. **"Cases accumulate into knowledge"** → Diagnostic Case Store (LanceDB) → similar search + statistics + resolution tracking

6. **"Korean models?"** → Solar Pro (AWS) + Exaone 3.5 (on-prem) → per-task optimal model assignment

Repeatedly asking "What's the weakness in this design?" at each step resulted in a 15-section, 3,800-line design document and 21 files of ~4,800 lines of implementation completed in a single session.

Key Design Decisions

Deterministic Engine with LLM on Top

95% of the agent runs on a Python rule engine (if-else, thresholds, pattern matching). 48 checklist items are auto-evaluated, generating reports in finding + likely_cause + suggested_action format. LLMs are used only for discussing "how to interpret these numbers" with operators.

The core reason for this separation: LLM non-determinism is a risk in audit contexts — same input may produce different diagnoses. The rule engine establishes facts; the LLM discusses interpretation of those facts.

Minority Report: "Missing a Signal is Worse than a False Alarm"

Minority opinions (1/3) in the 3-agent consensus are structurally preserved. Initially considered pure Delphi (sequential deliberation), but identified the problem of "later agents conforming to earlier opinions." Final design: Round 1 locks minorities via independent voting, Round 2 only strengthens arguments — minorities are never deleted.

The name was inspired by the film *Minority Report*: one of three precogs makes a different prediction.

Feature Reverse-Mapping Pipeline Completion

Four critical gaps in recommendation reason generation were fixed:
1. English fallback replaced with Korean default templates
2. 12+ missing feature prefix-to-group mappings added
3. ReverseMapper integrated as Level RM fallback in InterpretationRegistry
4. `generate_l1()` connected to InterpretationRegistry via 3-tuple enrichment

These fixes ensure L1 reasons include Korean interpretations with IG direction and task context.

Implementation Method: Parallel Sub-Agent Execution

Design document → implementation plan → Phase-by-phase Sonnet sub-agent parallel execution → main agent (Opus) review. Pre-req (4) → Phase 0 (3) → Phase 1+2 (5) → Phase 3+4 (4) → Phase 5 (2) → gap fixes (5), 6 rounds of parallel execution total.

Each round: `py_compile` + `yaml.safe_load` + interface contract verification + hardcoding scan — maintaining a rhythm of "build fast, verify thoroughly."

Deliverables

- **Design document**: 3,861 lines (Typst) + 1,168 lines (Markdown) + on-prem handoff 430 lines
- **Implementation**: 21 Python files, ~4,800 lines
- **Configuration**: agent.yaml + agent_tools.yaml (38 tools) + checklist.yaml (53 items)
- **Document updates**: Paper 2 both versions + typst 10 files + design 6 + guides 3 = 20+ files
- **Diagrams**: 3 Paper 2 placeholders → fletcher diagrams, 15 docs/typst ASCII → fletcher conversion

- **Translation**: 5 docs/typst/en tech_ref files Korean → English full translation

PaperClip Adoption (2026-04)

PaperClip (2026.3, GitHub 30K stars) has a "zero-human company" philosophy that conflicts with our principles, but three operational mechanisms were worth adopting:

1. **Heartbeat Pattern**: Agents wake periodically to run checklists, but return `HEARTBEAT_OK` and sleep again if nothing changed. Applied to our CP5 (5min), CP6 (1h) periodic checks.
2. **Budget Cap (Prepaid Debit Card Model)**: Per-agent monthly token limits, 80% soft warning, 100% hard stop. The key insight is **graceful degradation** — when budget is exhausted, only LLM calls are blocked while the rule engine continues. This naturally aligns with our "on-prem baseline operates without LLM" design.
3. **Full Tool Trace**: Every `ToolRegistry.call()` is automatically logged, enabling complete reproduction of "what tool calls produced this diagnosis."

LangMem's **prompt self-improvement** was deliberately not adopted — from an audit perspective, "who approved this prompt?" cannot be answered.

Memory Framework Adoption (2026-04, follow-up)

Immediately after PaperClip implementation, we reviewed other memory frameworks (Mem0, Zep/Graphiti, Letta, SuperLocalMemory). Again, we **did not adopt any framework wholesale** — only four patterns were selectively incorporated.

Zep/Graphiti Temporal Knowledge Graph

"What was customer A's state at 2026-03-15?" — answering this audit query requires the model version, features, and verdicts from that moment. Rather than joining distributed components, we built `TemporalFactStore` with a `(entity, attribute, value, valid_from, valid_to)` schema that resolves this as a single filter. Shares the same LanceDB instance as `DiagnosticCaseStore`.

SuperLocalMemory Mathematical Decay

"Is a 3-year-old drift resolution approach still valid?" — no. Recent cases are more relevant. But **deleting** old cases violates the 7-year retention requirement. Solution: preserve originals but adjust **search weights only** via $\exp\left(-\frac{\text{age}}{\tau}\right)$ with a 90-day half-life default. Solved with 30 LOC added to `DiagnosticCaseStore.search_similar()`.

Mem0 Fact Compression

`InterpretationRegistry` provides feature-level Korean interpretations, but lacked **customer narrative profiles**. Without facts like "this customer prefers deposits and is risk-averse" in the L2a prompt, Solar Pro generates reasons without context. We implemented `FactExtractor` rule-based — 15 rules defined in YAML config, safely evaluated via Python `eval()` with a sandboxed `__builtins__`. **Zero LLM calls**.

Letta Recall Memory

Solved the problem of `BedrockDialogSession` losing dialog history on session termination via DynamoDB-backed `DialogRecallMemory`. Past conversations are retrieved by embedding search (or keyword fallback) and injected into the system prompt.

A Critical Wiring Bug Found in Quality Audit

After implementation, an interface contract audit found **one critical bug**: `generate_l1()` retrieved `customer_facts` but only attached them to a local variable; they never reached the L2a path through SQS. The entire M-3 chain was a **silent no-op**. Fixed by re-querying facts in `get_best_reason()` and injecting them directly into the SQS context dict.

The lesson: **even when sub-agents implement individual files correctly, connecting data flows between files is a separate task**. Main agent's final verification is essential.

Future Plans

Academic and Industry Publications

- **arXiv Paper Submission**: 2 papers (system architecture paper + AI agent development methodology paper)
- **Anthropic Case Study Submission**: A case study on building a financial AI system using Claude Code
- **GARP Submission**: A paper combining the FRM credential with an AI risk management perspective

Regulatory Engagement

- **FSS AI Basic Act Compliance Review Request**: As enforcement decrees and guidelines for the AI Basic Act are being drafted, the team plans to request a review of this system's explainability framework at the appropriate timing.

Follow-up Work

- **On-premises Production Data Results**: Performance results from actual production data to be added as paper supplements
- **Public GitHub Repository**: A sanitized version of the codebase with organizational information removed will be published as a public repository

This project was completed not by overcoming a "lack of resources"
but by "redefining resources."
It demonstrates that one desktop GPU combined with AI agents
can substitute for dedicated infrastructure.